

Maestría en Economía
Facultad de Ciencias Económicas -- Universidad Nacional de La Plata

Seminario de Stata

clase 1

Santiago Garganta
(CEDLAS-UNLP)

Universidad Nacional de La Plata
2017

Contenido

- Curso introductorio de Stata (4 clases)
- En este curso se cubren temas referidos a la utilización de Stata como software estadístico/econométrico
- Evaluación: TPs y Examen domiciliario.
- Fechas de entrega: Examen (28/4 o 5/5)
- Dropbox?

Clase 1

- Introducción a Stata y su sintaxis
- Comandos básicos.
- Gestión de base de datos
- Organización de un proyecto en archivos .do y .log
- Algunos ejercicios introductorios

Descripción Stata

- El Stata es un paquete estadístico-econométrico que puede utilizarse para
 - manipulación de datos
 - análisis de datos (tabulados, gráficos, etc.)
 - econometría
 - además, es programable

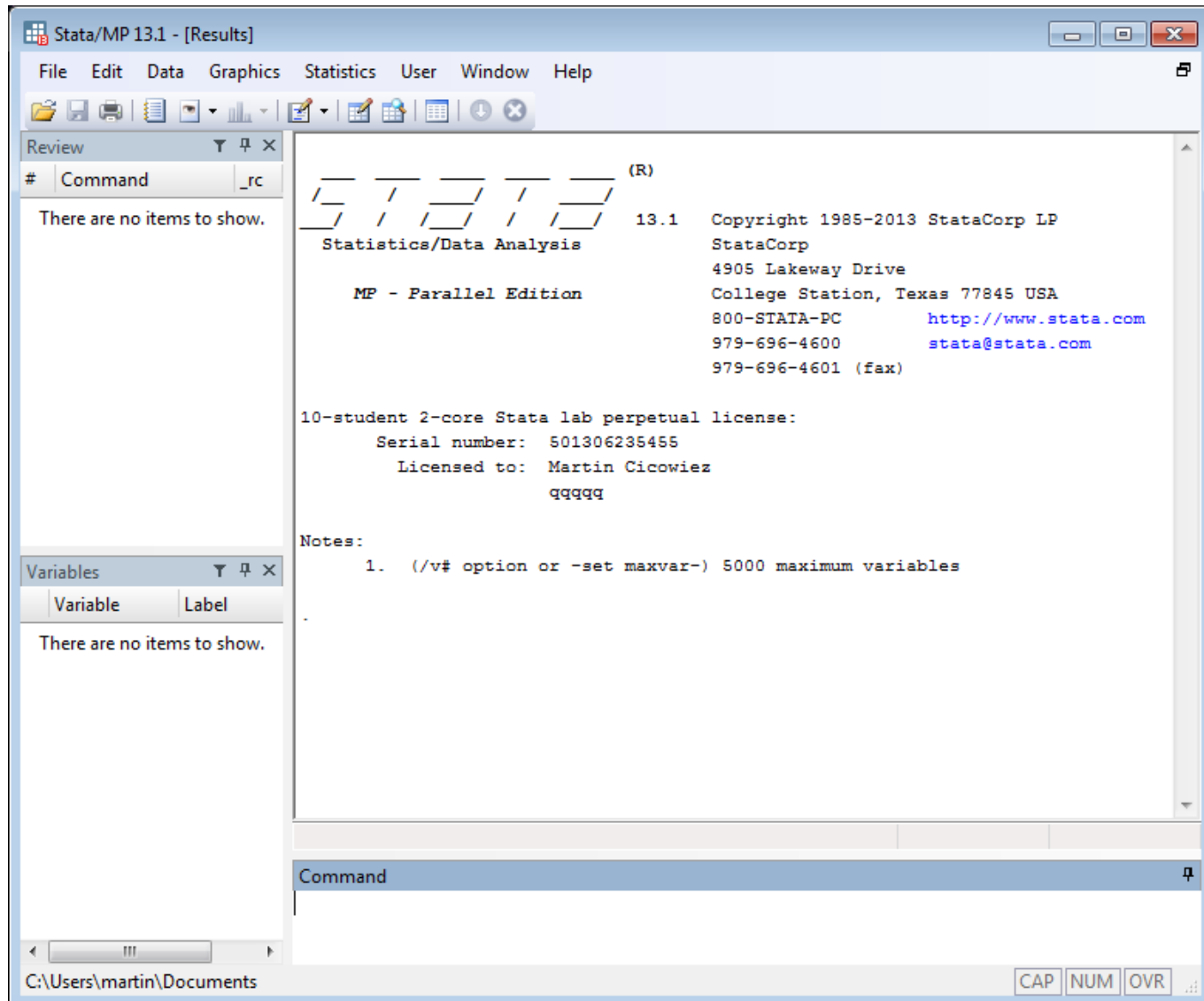
Utilización de Stata

- Interfaz Stata: comprende el entorno del trabajo que se realiza con el software.
- En primera instancia se trabajará con la interfaz de ventanas.
- Luego, interacción archivos .do y .log (ventanas siguen siendo útiles).
- Esto permite una utilización más eficiente del software.

Utilización Stata

- Es recomendable la lectura del manual impreso del Stata “User’s Guide”
 - Desde el “help” de stata se puede acceder al .pdf del “User’s Guide”
- Se sugiere configurar Windows para utilizar la coma (,) como separador de miles y el punto (.) como separador decimal
 - es particularmente útil cuando se importan datos a Stata; por ejemplo, desde Excel, Access, SPSS, etc.

La Interfaz de Stata



La interfaz de Stata

- Las diferentes ventanas que componen la interfaz son:
 - “**Variables**”: muestra las variables que están en la base de datos
 - “**Command**”: en esta ventana el usuario introduce los comandos
 - “**Results**”: muestra los resultados que se obtienen al aplicar los comandos
 - “**Review**”: muestra el historial de comandos utilizados

La Interfaz de Stata – cont.

- El Stata puede utilizarse mediante menús y cuadros de diálogo. Sin embargo, preferimos trabajar con la línea de comandos
 - escribiendo cada comando que queremos ejecutar
- Los comandos pueden introducirse
 - (a) de forma inmediata a través de la ventana Command, o
 - (b) a través de archivos de texto que Stata ejecuta de manera secuencial (archivos do)
- Los resultados que Stata muestra en la ventana Results pueden almacenarse en un archivo de texto (archivos log).

La Sintaxis del Lenguaje

La sintaxis básica del lenguaje que utiliza Stata es

```
[by varlist:] command [varlist] [=exp] [if exp]  
                                [in range] [weight]  
                                [using filename] [, options]
```

donde los corchetes indican que se trata de una parte opcional; cabe resaltar que el “help” incluido en Stata explica qué parte de esta sintaxis acepta cada comando.

Bases de Datos

- En el seminario se utilizarán diferentes bases de datos
 - Encuestas de hogares
 - Cuentas Nacionales
 - Datos experimentales
 - otros
- Diferentes ejemplos de comandos simples para ver cómo se utilizan las diferentes partes de la sintaxis. Luego, aplicación de ejercicios más extensos.

Bases de Datos

- A modo de ejemplo, en este ejercicio introductorio utilizaremos la Encuesta de hogares de Ecuador 2011 (ENEMDU)
- ¿qué es una encuesta de hogares?
- Es un relevamiento sobre los hogares que nos permiten conocer características socioeconómicas, laborales y de ingreso (consumo) de los mismos.

Primeros Comandos

- Los comandos que veremos a continuación se introducen de a uno por vez en la línea de comando del Stata.
- Luego, veremos cómo utilizar archivos para almacenar nuestro trabajo (.do)
- Utilizaremos pocas variables (base resumida procesada en CEDLAS)
 - ipcf = ingreso per cápita familiar
 - pondera = factor de expansión
 - hombre =1 si hombre
 - urbano =1 si urbano
 - region = varía según país

Primeros Comandos

- `* intro_stata.do`
- `clear all`
- `set mem 100m`
- `cd "C:\cedlas\2017\Seminario_Stata\clase1"`
- `cd`
- En primer lugar, nos ubicamos en la carpeta donde vamos a trabajar con el comando “cd ...”
- Este comando se puede aplicar también para saber dónde estamos ubicados o trabajando

set memory

- En Stata la base de datos se almacena en la memoria RAM (ventaja: ejecución de comandos rápida pero el tamaño de las base de datos se encuentra limitado por la memoria RAM de la pc)
- Para ampliar el espacio de memoria RAM que usa Stata:

set memory 200m (asigna 200 MB)

- Versión 13 de Stata no es necesario asignar memoria

Primeros Comandos

- `* apertura base de datos`
- `use "ecu11_cedlas_resumida.dta", clear`
- Se puede importar una base de datos en otro formato que no sea .dta (para ello, se utilizan otros comandos)
- Comando *describe*, detalle general de la base de datos
- `describe`
- Encuesta de Hogar: notar que hay variables que son individuales y otras a nivel de hogar.

Primeros Comandos

- **Comando “summarize”**
- Descripción estadística de una, varias o todas las variables de la base de datos. No considera los missing data (.)
- `summarize ipcf`
- `summarize ila ipcf`
- `summarize`
- `summarize p*`
- **Explicación, sintaxis del comando**
- `help summarize`

Abreviaturas

- Es posible utilizar abreviaturas para los comandos y variables
- Recomendación no abusar de las mismas
- summarize **su**
- comandos para realizar tablas

Primeros Comandos

- **[if exp]**: la aplicación de algunos comandos puede restringirse a alguna condición o grupos de condiciones.
 - `summarize ipcf if urbano == 1`
 - `summarize ipcf if urbano == 0`
- urbano es una variable dummy. Qué interpretación tiene la media de la misma?
 - `summarize urbano`

Primeros Comandos

- Sintaxis
 - Sentencias condicionales
 - == igual
 - != distinto
 - >, <, >=, <=
 - Operadores lógicos
 - & (ampersand) and
 - | or

Replace

- `replace itf = itf / 1000`
- `replace ipcf = 0 if urbano == 0`
- `replace ipcf = 0 if urbano == 0 & hombre == 1`
- `replace ipcf = 0 if urbano == 0 | hombre == 1`

Primeros Comandos

- **[in range]**: la aplicación de algunos comandos puede restringirse a un rango de observaciones

summarize hombre in 1/10

summarize hombre in -10/-1

- Dependiente del orden de la base de datos.

Primeros Comandos

- Ordenar la base de datos

sort id

gsort -id (de manera descendente)

gsort +id (igual a sort)

- Ordenar por varias variables

sort id edad

gsort id -edad

preserve y restore

- * preserve y restore
- summarize hombre
- preserve
- replace hombre=0 if hombre==1
- summarize hombre
- restore
- summarize hombre

Primeros Comandos

- `generate edad2 = edad*edad`
- `gen grupo = 1 if edad<=10 & hombre==1`
- `replace grupo = 0 if grupo==.`
- Cómo generar una variable que sea igual al `ipcf` normalizada por la media de `ipcf`

Primeros Comandos

- `summarize ipcf`
- `return list`
- `gen ipcf_media = ipcf/r(mean)`
- `rename ipcf_media ipcf_mu`

Otros Comandos (Tablas)

- **tabulate**, tabular diferentes valores de una variable (cantidad de observaciones)

tabulate region

- Se puede tabular más de una variable

tabulate region hombre

Otros Comandos (Tablas)

- **table**, tabla más flexible
- Se pueden combinar valores de una variable con estadísticos de otras referidos a los valores de la primera.
- `table region`
- `table region, c(mean ipcf)`
- `table region, c(freq mean ipcf sd ipcf)`
- `table region, c(freq mean ipcf mean ila)`

Ver los diferentes estadísticos (help)

Primeros Comandos

- **[by varlist:]** Permite particionar la base de datos en tantos grupos como valores de la variable existan.
- `by region: summarize ipcf`
- Es requisito ordenar la base de datos por la variable que se utiliza en el **by**, o
- `sort region`
- `by region: summarize ipcf`
- `bysort region: summarize ipcf`

Primeros Comandos

- En ciertos casos necesitamos más de una variable para particionar la base (utilizar la variable `ila`)

`sort region hombre`

`by region hombre: summarize ila`

- Resumir en una sola variable las seis categorías que surgen de las variables `region` y `hombre`. Aplicar el comando **`by`** con esa nueva variable

Primeros Comandos

- Resumir en una sola variable las seis categorías que surgen de las variables `region` y `hombre`.
Aplicar el comando **by** con esa nueva variable

```
egen reg_h = group(region hombre)
```

```
sort reg_h
```

```
by reg_h: summarize ila
```

- Comando **egen** muy útil, varias opciones

Otros Comandos

- Borrar observaciones y variables
- `drop if edad < 5`
- `drop ii`
- No es posible borrar estas dos dimensiones al mismo tiempo

Otros Comandos

- **[, opciones]** existen comandos que aceptan opciones adicionales (help es muy importante para esto)

summarize ipcf

summarize ipcf, detail

egen (más opciones)

- `gen suma1 = sum(pondera)`
- `egen suma2 = sum(pondera)`
- `egen suma3 = total(pondera)`

egen (más opciones)

- `gen aux = ila + inla`

egen (más opciones)

- `sort ila`
- `gen aux = ila + inla`
- `egen aux1 = rsum(ila inla)`

egen (más opciones)

- `sort ila`
- `gen aux = ila + inla`
- `egen aux1 = rsum(ila inla)`
- `egen aux2 = rsum(ila inla), missing`
- `summ aux*`
- `drop aux*`

La Utilización de Ponderadores

- El Stata permite distintos tipos de ponderación (help weight):
 - **fweights = frequency** (indican el número de observaciones duplicadas)
 - **pweights = sampling** (representan la inversa de la probabilidad de que la observación sea incluida de acuerdo al diseño de la muestra)
 - **awweights = analytic** (las observaciones representan promedios y los ponderadores son inversamente proporcionales a la varianza del mismo)
 - **iweights = importance** (indican la importancia de cada observación de acuerdo a algún criterio)
- En general, los comandos de Stata tienen asignado un tipo de ponderación por defecto; es el que utilizaremos.

La Utilización de Ponderadores

- **Sin ponderar**

summarize ipcf

- **Ponderado**

summarize ipcf [w=pondera]

Maestría en Economía
Facultad de Ciencias Económicas -- Universidad Nacional de La Plata

Seminario de Stata

clase 2

Santiago Garganta
(CEDLAS-UNLP)

Universidad Nacional de La Plata
2017

Clase anterior

- Introducción al programa
- Explicaciones generales (recomendaciones, sintaxis general, modo de trabajar en stata, ponderadores, etc.)
- Análisis de algunos comandos básicos (summarize, generate, replace, sort, preserve y restore, by, drop, table, tabulate y algunas funciones del comando egen)

Clase 2

- Algunas aclaraciones de los comandos analizados en la clase anterior.
- Finalizar con los comandos básicos (con la encuesta de hogares de Ecuador 2011) y resolver ejercicios iniciales.
- Organización del trabajo en archivos .do y .log
- Tipos de datos
- Gestión de base de datos

egen

- `gen suma1 = sum(pondera)`
- `egen suma2 = sum(pondera)`
- `egen suma3 = total(pondera)`

egen

- `gen suma1 = sum(pondera)`
- ~~`egen suma2 = sum(pondera)`~~
- `egen suma3 = total(pondera)`

egen (más opciones)

- `gen aux = ila + inla`

egen (más opciones)

- `sort ila`
- `gen aux = ila + inla`
- `egen aux1 = rsum(ila inla)`

egen (más opciones)

- `sort ila`
- `gen aux = ila + inla`
- ~~`egen aux1 = rsum(ila inla)`~~
- ~~`egen aux2 = rsum(ila inla), missing`~~
- `summ aux*`
- `drop aux*`

egen (más opciones)

- `sort ila`
- `gen aux = ila + inla`
- `egen aux1 = rowtotal(ila inla)`
- `egen aux2 = rowtotal(ila inla), missing`
- `summ aux*`
- `drop aux*`

La Utilización de Ponderadores

- El Stata permite distintos tipos de ponderación (help weight):
 - **fweights = frequency** (indican el número de observaciones duplicadas)
 - **pweights = sampling** (representan la inversa de la probabilidad de que la observación sea incluida de acuerdo al diseño de la muestra)
 - **awweights = analytic** (las observaciones representan promedios y los ponderadores son inversamente proporcionales a la varianza del mismo)
 - **iweights = importance** (indican la importancia de cada observación de acuerdo a algún criterio)
- En general, los comandos de Stata tienen asignado un tipo de ponderación por defecto; es el que utilizaremos.

La Utilización de Ponderadores

- **Sin ponderar**

summarize ipcf

- **Ponderado**

summarize ipcf [w=pondera]

Otros Comandos

- **display:** muestra operaciones o texto que sigue al comando
- `display "hola"`
- `display 2 * 2`
- `display ipcf[1]`
- `display ipcf[_N]`
- `display "Tasa de desempleo = " 2*4`

Ejercicio 1

- Calcular la cantidad de miembros por hogar promedio en Ecuador 2011

summarize miembros

???????

Ejercicio 1

- Calcular la cantidad de miembros por hogar promedio en Ecuador 2011

summarize miembros [w=pondera]

Por qué no?

Ejercicio 1

- Calcular la cantidad de miembros por hogar promedio en Ecuador 2011

sort id

egen hogar=tag(id)

summarize miembros [w=pondera] if hogar==1

Ejercicio 2

- Calcular la variable **miembros** utilizando los comandos analizados en clase. Nombrar a esta nueva variable **miembros1**

Ejercicio 2

- Calcular la variable **miembros** utilizando los comandos analizados en clase. Nombrar a esta nueva variable **miembros1**

gen a = 1

Ejercicio 2

- Calcular la variable **miembros** utilizando los comandos analizados en clase. Nombrar a esta nueva variable **miembros1**

gen a = 1

bysort id: egen miembros1=total(a)

Ejercicio 2

- Calcular la variable **miembros** utilizando los comandos analizados en clase. Nombrar a esta nueva variable **miembros1**

gen a = 1

bysort id: egen miembros1=total(a)

compare miembros miembros1

Organización de un proyecto

- Hasta el momento solo hemos trabajado con la ventana de comandos
- Archivos **.do**: archivos de texto que contienen toda la secuencia de comandos
- Los resultados que surgen de aplicar dicha secuencia los podemos almacenar en un archivo **.log**

Archivos do

- Los “do” son archivos de texto plano con código de Stata que se ejecuta de forma secuencial, línea por línea – programación
 - pueden editarse con el bloc de notas de Windows
 - no recomendamos el uso del editor de archivos do de Stata – al menos, hasta la versión 11
- Es recomendable trabajar siempre con archivos do – permiten replicar resultados fácilmente.

Archivos do

- Para los archivos .do existe un editor de Stata y también editores de texto externos (editplus, crimson, notepad++)
- Los editores externos contienen algunas funciones adicionales al de Stata.
- Utilizaremos el notepad++

Archivos do

- Ver intro_stata.do.
- Correr el .do
- Comentarios?

Ejercicio inicial (.do)

- Trabajar en la carpeta
“c:\Seminario_Stata\clase2”
- En esa carpeta está la base de datos
eph2010_resumida.dta
- Crear un archivo .do
- Para ello abrir el notepad++
- Guardar el archivo .do con el nombre
ejercicio1.do
- Recuerden agregarle la extensión .do cuando
guardan el archivo

Ejercicio inicial

- En este primer archivo .do:
 - Ubicarnos en la carpeta donde vamos a trabajar
 - Abrir la base de datos
 - Mostrar el contenido de la base de datos
 - Obtener la edad promedio de la muestra
 - Cerrar
- La mecánica es correr el do por primera vez y luego para los cambios posteriores utilizar la ventana Review

Ejercicio inicial

- Almacenar los resultados en un archivo **.log**
- Es importante saber que existe:
 - Pero vamos a estar almacenando los resultados que nos interesan (matrices, tablas, exportacion de resultados a archivos .xls)
 - Últimas clases
- En el archivo .log se guardan los resultados entre el comando **log using** y **log close**

Ejercicio inicial

- En el archivo .log se guardan los resultados entre el comando **log using** y **log close**
- El comando log using resulta en un error si ya existe un archivo log en uso o abierto
- Por eso debemos cerrar cualquier log que esté en uso (log close)
- El problema es que log close dará un error si no hay un log abierto para cerrar
- Introducción del comando capture (OJO!)

Base de datos

- La base de datos se corresponde con la Encuesta Permanente de Hogares 2010 (EPH), resumida
- Datos socioeconómicos, laborales e ingresos de los hogares.
- La base de datos es una tabla de doble entrada:
 - Las columnas son variables
 - Las filas son observaciones
 - En cada celda se pueden guardar diferentes tipos de datos (numéricos, palabras, fechas)

Datos

- Comando describe muestra el tipo de datos.
- Numérico, hay varios tipos
 - byte, integer, Long (enteros). Se diferencian por los límites
 - Float, Double (con decimales). Se diferencian no solo por la extensión sino por la cantidad de decimales

Datos

- Cuando genero una variable puedo aclarar el tipo de dato que quiero que se genere. Por ejemplo: **generate byte prueba = 0**
- Pero Stata cambia el tipo de dato de la variable numérica en función de los datos que se ingresen en la misma
- **replace prueba = 150**
- **replace prueba = 2.5**

Datos

- Stata trabaja con el punto como separador de decimal y coma separador de miles
- Asegurarse que la configuración regional de la pc se encuentre en línea con Stata para evitar inconvenientes
- Inicio | Panel de control | Configuración regional

Datos

- Variables String
 - generate str auxiliar = ""
 - replace auxiliar = "hola"
 - replace auxiliar = "8"
- Para transformar una variable string en numérica puedo aplicar el comando encode:
 - encode pais, gen(npais)
 - **npais** será una variable numérica con distintos valores según cada una de las palabras presentes en la variable **pais**

Datos

- Para transformar una variable string (con números) en numérica puedo aplicar el comando `destring`:

`destring auxiliar, replace`

- La variable `region` es numérica o string?

Datos

- Puedo transformar una variable numérica en string:
 - **generate reg_str = string(region)**
- Puedo transformar una variable string en numérica de la siguiente manera (alternativa a destring):
 - **generate reg_num = real(reg_str)**
 - Ver diferencias con
encode reg_str, gen(reg_num1)
- Si tengo string con palabras encode
- Si tengo string con numeros (ambas) pero ojo con las diferencias!

Tipo de datos

- Tipo de dato fecha
- Stata trabaja con fechas a través de un formato denominado “elapsed dates” (%d)
- La fecha se representa con un número que surge de considerar la diferencia en cantidad de días respecto al 1ro de enero de 1960
- 0 se corresponde con 1ro de enero de 1960
- -3 con 29 de diciembre de 1959
- Se pueden representar fechas desde el 1/1/100 al 31/12/9999

Tipo de datos

- Tipo de dato fecha
- Funciones para transformar datos con fechas en formato %d
 - date: permite convertir una variable string en %d
 - mdy: permite tomar tres argumentos (día, mes y año) y convertirlos en %d

Tipo de datos

- Ejemplo date:
 - Agregamos los datos de la fecha de nacimiento de algunos individuos (falsos)
 - Con diferentes formatos de fecha
 - Convertir esta información de fechas en string en elapsed date
 - Por qué %d? Es posible manipular mejor los datos (cálculos, orden, etc.)

Tipo de datos

- Ejemplo date:
 - Se genera una variable %d que representa la fecha de la variable string *fecha*
 - generate sdate = date(fecha,"DMY")
 - Ver Resultados
 - Se genera como resultado una variable numérica que almacena la diferencia de días con respecto al 1/1/1960
 - Se observa que la función date puede leer múltiples formatos de fechas (ver Help)

Tipo de datos

- Ejemplo date:
 - Hay una observación que stata no la puede interpretar (missing)
 - Por qué?
 - Se puede solucionar

Tipo de datos

- Ejemplo date:
 - generate sdate1 = date(fecha,"DM19Y")
 - generate sdate2 = date(fecha,"DMY",2030)
 - En el primer caso se indica la centuria o siglo por defecto en el caso de los años definidos con dos números
 - En el segundo caso se indica el año máximo de conversión

Tipo de datos

- Ejemplo date:
 - El formato elapsed date es útil para stata pero ilegible para la mayoría de la gente. Existe la posibilidad de dar formato a la visualización de una fecha
 - `format sdate2 %d`
 - `format sdate2 %dD_m_CY`
 - `format sdate2 %dD/m/Y`
 - `format sdate2 %dD/M/Y`
 - `format sdate2 %dD/n/Y`
 - `format sdate2 %dD/N/Y`

Tipo de datos

- Funciones para extraer información de una variable fecha (sdate1):

`year(sdate1)`

`month(sdate1)`

`day(sdate1)`

`halfyear(sdate1)`

`quarter(sdate1)`

`week(sdate1)`

`dow(sdate1)`

`doy(sdate1)`

Tipo de datos

- Ejemplo date:
 - Otra función para transformar variables en fechas con formato %d es mdy():
 - gen XX = mdy(mes, dia, anio)
 - gen XX = mdy(mes, dia, 1900+anio)
 - Partir de la variable sdate1

Maestría en Economía
Facultad de Ciencias Económicas -- Universidad Nacional de La Plata

Seminario de Stata

clase 3

Santiago Garganta
(CEDLAS-UNLP)

Universidad Nacional de La Plata
2017

Clase anterior

- Análisis de los comandos básicos y ejercicios iniciales simples.
- Organización del trabajo en archivos .do y .log
- Tipos de datos

Clase 3

- Análisis de gestión de base de datos
- Datos experimentales
- Regresiones
- Comandos post-regresión
- Macro locales

Tipo de datos

- Tipo de dato fecha
- Stata trabaja con fechas a través de un formato denominado “elapsed dates” (%d)
- La fecha se representa con un número que surge de considerar la diferencia en cantidad de días respecto al 1ro de enero de 1960
- 0 se corresponde con 1ro de enero de 1960
- -3 con 29 de diciembre de 1959
- Se pueden representar fechas desde el 1/1/100 al 31/12/9999

Tipo de datos

- Tipo de dato fecha
- Funciones para transformar datos con fechas en formato %d
 - date: permite convertir una variable string en %d
 - mdy: permite tomar tres argumentos (día, mes y año) y convertirlos en %d

Tipo de datos

- Ejemplo date:
 - Agregamos los datos de la fecha de nacimiento de algunos individuos (falsos)
 - Con diferentes formatos de fecha
 - Convertir esta información de fechas en string en elapsed date
 - Por qué %d? Es posible manipular mejor los datos (cálculos, orden, etc.)

Tipo de datos

- Ejemplo date:
 - Se genera una variable %d que representa la fecha de la variable string *fecha*
 - generate sdate = date(fecha,"DMY")
 - Ver Resultados
 - Se genera como resultado una variable numérica que almacena la diferencia de días con respecto al 1/1/1960
 - Se observa que la función date puede leer múltiples formatos de fechas (ver Help)

Tipo de datos

- Ejemplo date:
 - Hay una observación que stata no la puede interpretar (missing)
 - Por qué?
 - Se puede solucionar

Tipo de datos

- Ejemplo date:
 - generate sdate1 = date(fecha,"DM19Y")
 - generate sdate2 = date(fecha,"DMY",2030)
 - En el primer caso se indica la centuria o siglo por defecto en el caso de los años definidos con dos números
 - En el segundo caso se indica el año máximo de conversión

Tipo de datos

- Ejemplo date:
 - El formato elapsed date es útil para stata pero ilegible para la mayoría de la gente. Existe la posibilidad de dar formato a la visualización de una fecha
 - `format sdate2 %d`
 - `format sdate2 %dD_m_CY`
 - `format sdate2 %dD/m/Y`
 - `format sdate2 %dD/M/Y`
 - `format sdate2 %dD/n/Y`
 - `format sdate2 %dD/N/Y`

Tipo de datos

- Funciones para extraer información de una variable fecha (sdate1):

`year(sdate1)`

`month(sdate1)`

`day(sdate1)`

`halfyear(sdate1)`

`quarter(sdate1)`

`week(sdate1)`

`dow(sdate1)`

`doy(sdate1)`

Tipo de datos

- Ejemplo date:
 - Otra función para transformar variables en fechas con formato %d es mdy():
 - gen XX = mdy(mes, dia, anio)
 - gen XX = mdy(mes, dia, 1900+anio)
 - Partir de la variable sdate1

Unión de base de datos

- Comandos merge y append
 - Merge se agregan nuevas variables a una base de datos existente
 - Append agrega nuevas observaciones

Unión de base de datos

- Merge: unión de dos bases (master y using database)
 - **Caso 1:1** combinación de un elemento con un elemento
 - **Caso 1:m** combinación de un elemento con muchos elementos
 - **Caso m:1**
 - **Caso m:m** (completa las combinaciones, difícil en la práctica)
- merge 1:1 varlist using data2
- Ordenar las bases por varlist

Ejercicio

- Tenemos la base de datos de la EPH dividida en varias partes. Debemos unir esas partes (bases) para obtener la base de datos inicial (ver las diferentes bases)
- Base de datos inicial eph2010_resumida
- 4 bases (2 con datos individuales y 2 con datos a nivel hogar)
- Cómo se podría resolver?

Ejercicio

- Unir la base de datos eph2010_indiv1 con eph2010_hogar1
- Hacer lo mismo con la otra mitad de las observaciones (eph2010_indiv2 y eph2010_hogar2)
- Finalmente, append.

Ejercicio adicional

- Unir la base de datos eph2010_indiv1 con eph2010_indiv2
- Hacer lo mismo con las observaciones a nivel hogar (eph2010_hogar1 y eph2010_hogar2)

Ejercicio adicional

- Unir la base de datos eph2010_indiv1 con eph2010_indiv2
- Hacer lo mismo con las observaciones a nivel hogar (eph2010_hogar1 y eph2010_hogar2)
- Finalmente, merge.

Otros comandos

- De gestión de base de datos. Para reestructurar la organización de la base de datos
 - xpose: permite transponer una base de datos
 - reshape: permite reorganizar datos contruidos en forma de columna en varias columnas o viceversa (wide o long)
 - collapse: permite crear una nueva base de datos con estadísticas agregadas
- Organización modular del proyecto

Otros comandos

- Organización modular del proyecto
 - master.do: se encuentra la estructura principal del proyecto
 - prepara_base.do: prepara la base de datos a utilizar
 - gestión_bases.do: aplica los ejercicios con los nuevos comandos

Ejercicio

- Primero creamos un master.do
- En prepara_base.do importamos una base de datos que no tiene el formato de stata (.dta)
- inc_region.csv
- insheet using..... (.csv, .txt, ver Help)
- Este comando se puede seguir utilizando pero ha sido reemplazado en stata por **import delimited**
- Guardar la base como .dta

Ejercicio

- Crear un nuevo .do (gestion_bases.do) que abra esta nueva base de datos
- Transformar la variable region en numérica
- Transponer la base de datos
- Volver a la base de datos original
- Etiquetar la variable region y los valores de la variable region

Ejercicio

- Comando reshape: permite configurar la base de datos bajo dos estructuras llamadas “wide” y “long”
- La base de datos se encuentra bajo una configuración ***wide***
- Cambiar la configuración a ***long***

Ejercicio

- Formato wide, hay una fila por region y 4 variables de ingreso sobre diferentes periodos.
- El formato long en este caso sería que cada observación sea las regiones en los diferentes momentos del tiempo (ij)

Ejercicio

- Comando collapse: permite crear una nueva base de datos con estadísticas resumidas
- Ejemplo:
 - Partir de la base de datos en formato long
 - Obtener una nueva base de datos que contenga la variable región y el valor promedio de *ila* en el tiempo en cada una de las mismas.
 - Existen otros estadísticos para agrupar los datos (ver help collapse)

Datos experimentales

- Stata permite crear datos experimentales
- En este caso los vamos a utilizar para luego aprender el manejo de algunos comandos y luego regresiones
- En primer lugar se debe indicar la cantidad de observaciones que va a contener la base de datos
- Luego, se generan las variables

Datos experimentales

- Construir una base de datos con 150 observaciones
 - clear
 - set obs 150
- Crear una variable de tipo fecha que contenga la identificación de días desde el 18/05/1983 en adelante
 - gen fecha = date("18/05/1983","DMY")
 - ???

Datos experimentales (Ej. 2)

- Construir una base de datos con 1000 observaciones
- Generar dos variables aleatorias distribuidas de manera uniforme (x, w)
- Generar otra variable aleatoria (u) cuyos valores surgen de una distribución Normal (0,1)
- Fijar la semilla antes de crear variables aleatorias para luego poder replicar los resultados

Datos experimentales

- Generar $y = 4 + 2.6 * x + 1.1 * w + u$
- Borrar u
- Regresión y x (y x w)
- Comando regress indica que la primera variable representa el término dependiente del modelo y las siguientes son los términos independientes

Maestría en Economía
Facultad de Ciencias Económicas -- Universidad Nacional de La Plata

Seminario de Stata

clase 4

Santiago Garganta
(CEDLAS-UNLP)

Universidad Nacional de La Plata
2017

Clase anterior

- Análisis de gestión de base de datos
 - merge
 - append
 - reshape
 - Collapse
- Datos experimentales

Clase 4

- Datos experimentales
- Regresiones
- Comandos post-regresión
- Macro locales
- Matrices
- Bucles
- Argumentos (organización modular)
- Ejercicios

Datos experimentales

- Stata permite crear datos experimentales
- En este caso los vamos a utilizar para luego aprender el manejo de algunos comandos y luego regresiones
- En primer lugar se debe indicar la cantidad de observaciones que va a contener la base de datos
- Luego, se generan las variables

Datos experimentales (clase anterior)

- Construir una base de datos con 150 observaciones
 - clear
 - set obs 150
- Crear una variable de tipo fecha que contenga la identificación de días desde el 18/05/1983 en adelante
 - gen fecha = date("18/05/1983","DMY")
 - ???

Datos experimentales (Ej. 2)

- Construir una base de datos con 1000 observaciones
- Generar dos variables aleatorias distribuidas de manera uniforme (x , w)
- Generar otra variable aleatoria (u) cuyos valores surgen de una distribución Normal (0,1)
- Fijar la semilla antes de crear variables aleatorias para luego poder replicar los resultados

Datos experimentales

- Generar $y = 4 + 2.6 * x + 1.1 * w + u$
- Borrar u
- Regresión y x (y x w)
- Comando regress indica que la primera variable representa el término dependiente del modelo y las siguientes son los términos independientes

Datos experimentales

- Comandos Post regresión
 - **regress y x w**
 - **predict yhat** (permite obtener el valor estimado de y de acuerdo al modelo de regresión)
 - Se puede obtener lo mismo de la siguiente manera:
gen yhat1=_b[_cons] + _b[x]*x + _b[w]*w
 - Los operadores **_b[]** permiten acceder a los valores estimados de los parámetros de la regresión

Datos experimentales

- Comandos Post regresión
 - El comando **predict** también nos permite obtener los errores de la regresión
 - **predict** errores, resid
 - Se puede obtener lo mismo de la siguiente manera:
gen errores1=y – yhat
 - Comparar

Datos experimentales

- Comandos Post regresión
 - El comando **test** permite realizar pruebas de hipótesis lineales sobre los parámetros estimados de la regresión
 - Ejemplo: test x=0
 - (1) x = 0
 - F(1, 997)=548.31
 - Prob > F=0.0000
 - Esto indica que se rechaza la Hipótesis nula de que el coeficiente de x=0

Datos experimentales

- Comandos Post regresión
 - Esto indica que se rechaza la Hipótesis nula $x=0$
 - Este resultado ya se encontraba en la salida de regresión. Observar que el estadístico t de x en la salida de regresión es 23.42 (valor correspondiente a una distribución t). Si la distribución F es una t al cuadrado, podemos comprobar que 23.42 al cuadrado es aproximadamente 548
 - Otros test: test $x=2.5$, test $x+w=3.7$

Datos experimentales

- Grafico:
 - regress y x
 - Obtener el y estimado
 - Graficar

- regress y x w
- Obtener el y estimado
- Graficar

Macros Locales: Una Introducción

- Ejercicio:
 - generar dos variables aleatorias x y y distribuidas uniformemente entre 0 y 1
 - generar una tercera variable como
$$z = (x + y) / (\mu_x + \mu_y)$$

Ejercicio

- Se pide (utilizando EMNV 98 de Nicaragua):
 - calcular la tasa de pobreza empleando una LP de 324.964 (=2.5 US\$ diarios a PPA)
 - Utilizar una macro local para la línea de pobreza

Ejercicio

- Se pide (utilizando EMNV 98 de Nicaragua):
 - calcular la tasa de pobreza empleando una LP de 324.964 (=2.5 US\$ diarios a PPA)
 - Utilizar una macro local para la línea de pobreza
 - Hacer el mismo cálculo para la zona rural y urbana
 - Mostrar los resultados con una tabla

Ejercicio

- Resultado
 - Pobreza Total: 48.87%
 - Pobreza Urbana: 37.86%
 - Pobreza rural: 61.88%

Include vs do

- El comando `do` ejecuta un archivo `.do`
- El comando `include` representa un copy-paste de lo que contiene un archivo `.do`. Es decir, el contenido del último al archivo `.do` pertenece al `.do` donde se ejecuta el comando *include*
- Ver diferencias (relevancia de las mismas cuando se trabaja de forma modular)

Matrices

- Muy útiles para acumulación y visualización de resultados
- Crear matriz vacía
- Crear matriz con su contenido
- Asignar valores a celdas particulares de la matriz
- Operaciones con matrices
- Vincular matrices
- Eliminar matrices

Ejercicio de Pobreza (Nicaragua)

- Cálculo de la tasa de pobreza en Nicaragua
- Tasa de Pobreza Total, Urbana y Rural
- Acumular los resultados en una matriz
- Mostrar los resultados contenidos en la matriz
- Se puede exportar la matriz en otro formato (.xls)
- Antes se crean variables a partir de la matriz (svmat) y luego se exporta.
- Ver mkmat

Bucles (Loop)

- Nos permiten realizar un mismo calculo varias veces (iterando sobre diferentes valores o elementos)
- Entre {} expreso lo que se repite. Qué y cuantas veces lo voy a repetir lo determina el tipo de bucle
- Diferentes tipos de bucles:
 - foreach
 - forvalues
 - While
- Ver Ejemplos

Ejercicio a

- El siguiente ejercicio combina los conceptos analizados anteriormente:
 - Utilizar la base nicaragua 1998
 - Calcular el ingreso laboral promedio en cada region (4 en total)
 - Acumular los resultados en una matriz
 - Mostrar resultados

Ejercicio b

- El siguiente ejercicio combina los dos conceptos analizados anteriormente:
 - Generar una matriz de 2x3 (elegir los valores)
 - Mostrar el valor de la celda [2,3]
 - Mostrar todos los valores iterando por fila y columna